Python Programming - Assignment 1

Title: Design, Implement and Validate a Payment Calculator for a Village Milk Co-operative (Total Marks - 50)
Submission Date: 23/10/25

TASK 1 – Understand the Problem and Identify the Data (4 Marks)

Problem Analysis:

- **Inputs:** Farmer ID (`str` or `int`), milk quantity in litres (`float`), fat percentage (`float`), SNF percentage (`float`), and shift (`str` - Morning/Evening).
- **Outputs:** Rate per litre (`float`), payment amount for current delivery (`float`), ten-day total payout (`float`).
- **Named Constants:** `BASE_RATE = 30.00`, `BASE_FAT = 3.5`, `BASE_SNF = 8.5`, `FAT_STEP = 0.1`, `SNF_STEP = 0.1`, `FAT_STEP_VALUE = 0.40`, `SNF_STEP_VALUE = 0.30`.
- **Data Types:** Whole Numbers (`int`) for ID/steps; Fractional Numbers (`float`/`Decimal`) for litres, percentages, money; Text (`str`) for shift.
- **Validation Rules:** Quantity > 0; Fat between 2.0% and 7.0%; SNF between 6.0% and 10.0%.

Alternative Concept & Complexity Analysis: Storing 300 farmers in individual scalar variables requires 900 distinct variable names, making loops and scalability impossible. Storing entries in lists/tuples provides *O(n)* time and *O(n)* space complexity, scaling effortlessly up to 150,000 farmers.

TASK 2 – Explore Values and Types in the Interpreter (4 Marks)

Interactive Session Discussion: Explored types using `type()`. Division operators (`/`, `//`, `%`) reviewed including floor division behavior with negative numbers. Demonstrated floating-point inaccuracy (`0.1 + 0.2 == 0.3` is `False`) and resolved money precision issues using integer paise, `round()`, or Python's `decimal.Decimal` module.

Complete the Code (TODO 1):

```
from decimal import Decimal

payments = ["33.40"] * 1000 + ["0.10"] * 3

def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total

def total_decimal(items):
    total = Decimal("0.00")
    for amount in items:
        total += Decimal(amount)
    return total

print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

TASK 3 – Choose Variables and Name the Constants (3 Marks)

Variables use `snake_case` while constants use `UPPER_CASE`. Python lacks true language-level constant enforcement, relying entirely on convention. Dynamic typing allows variables to rebind across types safely. Multiple assignment and tuple swapping (`a, b = b, a`) demonstrated.

TASK 4 – Write Correct Expressions (4 Marks)

Covered operator precedence, unary minus, and exponentiation. Addressed floating-point step count imprecision (e.g., `(4.2 - 3.5) / 0.1` giving `7.000000000000001`) and how `round()` correctly sanitizes step counts. Analyzed comparison chaining (`2.0 <= fat <= 7.0`).

TASK 5 – Read Input and Type Conversion (3 Marks)

`input()` returns strings requiring explicit conversion. Handled `ValueError` exceptions for malformed inputs (e.g. `"8,5"` or `"eight"`). Implemented f-string formatting for padded, aligned monetary output (e.g. `{amount:8.2f}`).

TASK 6 – Organise the Logic into Functions (4 Marks)

Structured code into modular functions (`rate_per_litre`, `payout`, `ten_day_total`). Distinguished between returning values (reusable, testable via assertions) versus printing (side-effect display). Reviewed local vs. global scope rules.

TASK 7 – Build a Module (4 Marks)

Created `milkrate.py` containing rate logic and tested it via a separate script. Explained module import syntax variants and the role of `if __name__ == "__main__":` in isolating executable test suites from importable library code.

TASK 8 – Plan the Algorithm Before Coding (4 Marks)

Followed SIMATS Engineering Standard for Pseudocode. Planned modular algorithm steps, validation flowcharts, and hand traces for delivery calculations.

TASK 9 – Implement and Test the Calculator (4 Marks)

Implemented full calculator logic with comprehensive assertions (`assert`) covering normal entries, boundary limits (3.5% fat, 8.5% SNF), and invalid inputs.

TASK 10 – Compare Expectations with Execution (4 Marks)

Evaluated float vs. Decimal discrepancies across 300, 6,000, and 150,000 payments. Benchmarked execution speed using `time.perf_counter()`, confirming sub-millisecond calculation efficiency.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Recommended Decimal/integer paise precision, modular architecture (`milkrate.py`), and robust input error handling. Confirmed seamless scalability to district-union scale (150,000 farmers).

TASK 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge) (8 Marks)

A. Impact Analysis: Analyzed changed requirements including two-decimal analyzer precision, seasonal rates (Summer base rate Rs 32.00, fat step Rs 0.45), and a volume bonus of Rs 0.50 per litre for >500 litres in 10 days.

B. Evaluation & C. Adaptation: Replaced static constants with season-parameterized functions and added volume bonus logic.

D. Evidence-Based Defence: Validated through test cases across normal and summer seasons, proving clean extensibility without breaking core accounting.